1 Overview

MovieRama currently runs its data infrastructure on self-managed PostgreSQL and MongoDB instances. This document proposes a migration strategy to AWS cloud-native managed services.

PostgreSQL → Amazon RDS PostgreSQL

MongoDB → Amazon DocumentDB

2 Assessment phase

2.1 What to Inventory

- **Database sizes**: total data volume, per table and per collection. It will determine migration duration, instance size and storage costs on AWS.
- **Schema analysis**: all tables, relationships and index definitions. Indexes, relationships may need to be recreated manually on RDS/DocumentDB.
- **Connection audit**: all services connecting to PostgreSQL and MongoDB. Every service needs its connection string updated after migration.
- **User handling**: all existing users and their privileges. Users and roles must be recreated on the new managed instances.

2.2 What to Measure (Baseline)

- **Query latency**: how fast queries are running. We need to compare those numbers before and after in order to identify issues that might have arisen regarding the speed.
- **Throughput**: how many queries per second. Important to identifying the instance sizes
- **Error rates**: how often database connections fail. Another metric to understand if the migration has produced issues
- **Current uptime**: what our availability looks like today vs before the migration. Compare this with the SLO that we need to have

3. Target Architecture

The target architecture replaces self-managed instances with AWS fully managed services. The logical structure stays the same but the operational burden moves to AWS.

PostgreSQL → Amazon RDS PostgreSQL

We move from a self-managed primary/replica setup to Amazon RDS PostgreSQL with Multi-AZ deployment. AWS automatically manages failover, backups, patching and replication.

MongoDB → Amazon DocumentDB

We move from self-managed MongoDB to Amazon DocumentDB which is MongoDB-compatible — meaning minimal application code changes are needed. DocumentDB runs as a cluster with a primary and 2 replicas, with automatic failover handled by AWS.

4. Migration Strategy

The key principle is: never do a big-bang cutover. We migrate in phases, each one reversible. If anything goes wrong at any phase, we can roll back without impacting users.

Phase 1 — Preparation

We provision the AWS infrastructure using Terraform — RDS for PostgreSQL and DocumentDB for MongoDB. We recreate all users, privileges and verify all application connection strings. No data has been moved yet.

Phase 2 — Initial Data Load

We load a full snapshot into the new instances. For PostgreSQL we use AWS Database Migration Service (DMS). For MongoDB we use mongodump/mongorestore. Once complete we verify data integrity. Old databases remain primary.

Phase 3 — Continuous Replication

We enable Change Data Capture (CDC) on DMS so every new write is replicated to the new database in real time. We monitor replication lag and compare query results between old and new databases.

Phase 4 — Cutover

During the lowest traffic window we stop writes briefly, wait for replication lag to reach zero, then switch connection strings to the new instances. We monitor error rates and latency against our baseline. If anything looks wrong we roll back immediately.

Phase 5 — Decommission

We keep old databases running for a few weeks as a safety net. Once stable we decommission them.

5. SLO Continuity During Migration

Phases 1 to 3 run with zero user impact — old databases remain primary. The only risk window is Phase 4 — the cutover.

During cutover we monitor query latency and error rates against our baseline. If anything looks wrong we revert connection strings back to the old databases immediately.

After migration, RDS and DocumentDB handle patching and failover automatically — reducing downtime and improving our SLO compared to today.